AgriAgent Team Tasks until 21 October

Step-by-step instructions for Sahil, Sumant and Amit · hard deadline Wednesday 21 October, 23:59

AgriAgent Final Year Project · 17 October 2026

Read this first. Chaitanya and Claude finished Week 3 and part of Week 4 on 17 October: the real pipeline runs on the team laptop, the fake evaluation labels were replaced with official ones, and all eight Cycle 0 bugs were fixed, re-tested and closed. What happened and why is in `docs/AgriAgent_Week3_Dossier.pdf`. **This document tells you exactly what to do next.** Every task has steps, what to submit, and when it counts as done. You do **not** write Typst or code: you run commands, look at results, test the app, and fill in sheets. Chaitanya turns your sheets into the report.

Deadline and workload. Everything here is due Wednesday 21 October, 23:59. Post a short progress note in the WhatsApp group on **Monday 19 October by 21:00** (done / not done / blocked). Sahil's and Sumant's Week 3 deliverables were not submitted, so their lists also carry about **20% of their Week 5 work**. Amit's list is his Week 4 testing work plus about 10% of Week 5. If you are blocked for more than two hours, say so in the group with the exact error message; do not wait.

1 Where to put your work

What	Where
Screenshots, photos, videos	Google Drive folder **AgriAgent Evidence**, sub-folder with your name, one folder per task ID (for example `Sahil/SA-1`). Chaitanya shares the link in the group.
Sheets (results, test cases, label checks)	Google Sheet **AgriAgent Test Results**, one tab per task ID. Use exactly the columns given in the task.
Text content for the report	Google Doc **AgriAgent Report Content**, one heading per task ID. Plain bullet points are fine.
A defect you found	A GitHub issue (template in Section 7). One defect per issue.
Progress	WhatsApp: Monday 19 Oct by 21:00 and Wednesday 21 Oct by 23:59.

2 Day 0 for everyone (Saturday 18 – Sunday 19 October): set up, then understand

2.1 E-1 · Set up the project on your laptop (about 1.5 h)

1. Open a terminal (Windows: **Command Prompt**). If you already have the repository, go into it and run `git pull`. If not: `git clone https://github.com/Chaitanya-666/agriAgent.git` then `cd agriAgent`.
2. Create a Python environment (Python 3.11 or 3.12): `python -m venv .venv`
3. Activate it. Windows: `.venv\Scripts\activate` · Linux/macOS: `source .venv/bin/activate`
4. Install: `python -m pip install -e ".[dev]"` (takes 15–20 minutes the first time).
5. Copy the settings file. Windows: `copy .env.example .env` · Linux/macOS: `cp .env.example .env`
6. Create **your own** free Groq key at `console.groq.com/keys`. Open `.env` in a text editor and put it after `GROQ_API_KEY=` (no spaces, no quotes). Never post or commit this file.
7. Run `python scripts/check_providers.py groq`. You should see `[PASS] groq`.
8. Run the tests: `python -m unittest discover tests`. The last line must be `OK`.

9. Start the app: `streamlit run app/app.py`. A browser tab opens; you should see a yellow **MOCK OUTPUT** banner and four number cards.

Submit: one screenshot of the `[PASS] groq` line (hide the key), one of the test `OK` line, one of the app with the MOCK banner, in `Drive/<name>/E-1`. **Done when:** all three are there.

2.2 E-2 · Recollect and understand the project (about 3 h)

Read, in this order: `docs/AgriAgent_Week3_Dossier.pdf` (all), `docs/AgriAgent_Project_Map.pdf`, `docs/guides/01_Concepts.pdf`, `docs/guides/02_Codebase_Walkthrough.pdf` §10 (known issues K1–K12), and the questions for your own track in `docs/guides/03_Viva_Defence.pdf`.

Then answer these ten questions **in your own words** (3–5 lines each) under heading E-2 in the Report Content doc. The viva will ask the same things.

1. Name the five stages of the pipeline. Which ones are AI models and which are plain rules?
2. Grounding DINO labels a box "weed". Did it understand that the plant is a weed? Explain where the label comes from.
3. A weed mask has detection confidence 0.60. Is it sprayed? Which file and which rule decides?
4. Why was `eval_manifest_v1` thrown away, and what replaced it?
5. What does "crop pixels on the spray map" measure, and why can we only measure it on sugar beet images?
6. How can you tell on screen whether a result is REAL or MOCK? Why does it matter?
7. What does System 2 do, which model does it call today, and what happens when that call fails?
8. Why did one GeoJSON export contain 1,394 spray zones for 3 weeds, and what was changed?
9. The best published zero-shot method (Chong et al., IROS 2025) reaches about 12% weed IoU, a supervised network about 63%. What does that tell us about our own results?
10. List your own tasks in this document with their "done when" lines.

Done when: all ten answers are in the doc by Sunday 19 Oct, 23:59.

3 Sahil · validation tester (Week 4 + about 20% of Week 5)

SA-1 · Check the new evaluation labels by eye (T3.4)

about 3 h

Why: Every number we report is measured against these labels. A wrong label means a wrong number.

Steps:

1. Make a free Kaggle account. Settings → API → **Generate New Token**; save the token text in a file named `access_token` inside a folder `.kaggle` in your home folder (Windows: `C:\Users\<you>\.kaggle\access_token`).
2. In the activated environment: `python -m pip install kaggle`
3. Build the evaluation set (downloads about 110 MB): `python scripts/build_eval_manifest_v2.py`
4. Draw the labels: `python scripts/show_manifest_labels.py`. Pictures appear in `data/outputs/manifest_labels/` (green box = crop, red box = weed).
5. Open each of the 38 pictures. For each, fill one row in tab **SA-1**.

Submit: Tab SA-1 with columns: `image_id` · `every crop plant has a green box (yes/no)` · `every visible weed has a red box (yes/no)` · `any box on bare soil (yes/no)` · `notes`.
Done when: 38 rows filled; every "no" has a note saying which plant.

SA-2 · Hand-compute three crop-protection examples (for T3.5 tests)

about 1 h

Why: The new metric code will be tested against numbers you compute by hand, so a coding mistake cannot hide.

Steps: Each example is a 5 × 5 image. **Truth:** `C` = crop pixel, `W` = weed pixel, `.` = soil. **Spray:** `1` = sprayed, `0` = not sprayed. For each example compute: (a) crop protection failure = crop pixels sprayed ÷ all crop pixels, (b) weed recall = weed pixels sprayed ÷ all weed pixels, (c) herbicide savings = 1 − sprayed pixels ÷ 25. Show your working.

Example A

Truth:

```
C C . . .
C C . W .
. . . W W
. . . . .
W . . C C
```

Spray:

```
1 0 0 0 0
0 0 0 1 0
0 0 0 1 1
0 0 0 0 0
0 0 0 1 0
```

Example B

Truth:

```
. C C . .
. C C . .
. . . . W
W . . . .
. . . . .
```

Spray:

```
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
```

Example C

Truth:

```
C . . . W
C . . . W
. . W . .
. . . . .
. C . . .
```

Spray:

```
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
```

Submit: Photo of your handwritten working in `Drive/Sahil/SA-2`, and the nine results in tab SA-2.
Done when: Nine numbers, each with working.

SA-3 · Validation test plan: 15 cases

about 3 h

Why: The report needs a test chapter; validation asks “is the output correct?”, not “does the app run?”.

Steps: In tab **SA-3** write one row per case with columns `ID` · `what is checked` · `input` · `expected result` · `how measured` · `can run now (yes/no)`. Cover at least these 15: V-01 labels match plants (SA-1) · V-02 crop-protection formula (SA-2) · V-03 weed recall formula · V-04 herbicide savings formula · V-05 a crop-only image must give 0% crop sprayed · V-06 a blank image must give 0 sprayed zones in real mode · V-07 a low-confidence weed (below 0.70) must not be sprayed · V-08 a label mixing weed and crop words is withheld · V-09 cotton images are excluded from crop-protection numbers · V-10 the same image twice gives the same result · V-11 System 2 prescription matches the declared crop · V-12 no herbicide without a real prescription · V-13 crop-or-weed patch answers are compared with official labels · V-14 every reported number states model, device and REAL · V-15 savings never above 100% or below 0%. For cases you can run now (V-01, V-02, V-10, V-15 in the app with the sample photo), run them and add `result` and `evidence` columns.

Submit: Tab SA-3.
Done when: 15 rows; every “can run now = yes” row has a result and a screenshot in `Drive/Sahil/SA-3`.

SA-4 · Judge the 20 crop-or-weed patches blind (T2.2 follow-up)

about 1 h

Why: We compared three back-ends on 20 patches (Groq 16/20, Clef Flash 14/20, a row rule 13/20). A human baseline tells us how hard the patches really are.

Steps:

1. Chaitanya puts the 20 patch images (`data/outputs/t2_2/*.png`) in `Drive/shared/t2_2_patches` **without** the answers.
2. For each patch write `crop`, `weed` or `can't tell` **before** looking at any answer. Use only the picture.
3. Then open `docs/images/t2_2_patches.jpg` (answers) and fill the last two columns.

Submit: Tab SA-4: `patch` · `your answer` · `true answer` · `you were right (yes/no)` · `what made it hard`.
Done when: 20 rows; your score posted in the group.

SA-5 · Pick the hard test images for weekly escalation (Week 5 work)

about 2 h

Why: Each week we make the test harder: more images and harder cases. You choose the hard cases.

Steps:
1. After SA-1's build step, open the 120 sugar beet images in `data/datasets/sugarbeets_weeds_official/data/train/image/` with any image viewer.
2. Pick 15 images that are hard for weed-versus-crop, at least 3 of each kind: weed touching or under a crop plant · very small weeds · strong shadow or glare · crop plant cut off at the image edge · many weeds close together.
3. Do not pick images that are already in `eval_manifest_v2` (the list is the `image_id` column of SA-1).

Submit: Tab SA-5: `file name` · `kind of difficulty` · `one sentence why`.
Done when: 15 rows, at least 3 per kind.

SA-6 · Report content: data and evaluation (Week 5 work)

about 2 h

Why: Chaitanya writes the chapter; you collect the facts.

Steps: Under heading SA-6 in the Report Content doc, write bullet points for: the two datasets (name, source link, number of images, image size, which labels exist); why `eval_manifest_v1` was rejected (from the dossier §3.2, in your words); how `eval_manifest_v2` was built (selection rule); the three metrics (crop protection, weed recall, herbicide savings) with the formulas from SA-2; the dataset bias (beet weeds median 16 px, crops 68 px) and why it matters.

Submit: Heading SA-6 in the Report Content doc.
Done when: All five bullets present, with sources for every number.

4 Sumant · system tester and report content lead (Week 4 + about 20% of Week 5)

SU-1 · Department report format (Week 3 T2.6, overdue)

about 1 h

Why: The final report must follow the department format; without it the report cannot start.

Steps: Get the official report format (template file or written rules: margins, font, chapters, front pages, citation style) from the department or a senior batch, and upload it.

Submit: The file or photos in `Drive/Sumant/SU-1`; one message in the group.
Done when: Chaitanya confirms he has what he needs.

SU-2 · Check our herbicide table against the ICAR bulletin (T2.4, K12)

about 3 h

Why: The app recommends herbicides from a table someone typed by hand. If it is wrong, the app gives wrong chemical advice.

Steps:
1. Open `data/knowledge_base/icar_dwr_herbicide_use_bulletin22.pdf`.

2. Find the cotton herbicide table (Table 4.3, around pages 34–35) and the banned/withdrawn herbicide table (Table 3.3, around page 18). Write down the real page numbers you see.
3. For 5 cotton weeds (start with Trianthema portulacastrum, Parthenium hysterophorus, Echinochloa colona, Digera arvensis, Cyperus rotundus) copy from the PDF: herbicide, dose in g a.i./ha, water volume, waiting period, page number.
4. Open `src/agronomy/knowledge_store.py` on GitHub and search (Ctrl+F) for `Table 4.3`. Compare each row with what you copied. Do the same for the banned list: search `BANNED` in `src/agronomy/prescription.py`.

Submit: Tab SU-2: `weed` · `herbicide (PDF)` · `dose (PDF)` · `water (PDF)` · `waiting period (PDF)` · `page` · `same in our code? (yes/no)` · `difference`. One GitHub issue per difference.
Done when: 5 weeds and the banned list checked; every difference filed as an issue.

SU-3 · System test cycle 1: new cases (mock mode)

about 3 h

Why: The app changed on 17 October. These cases check the new behaviour like a user would.

Steps: Start the app (`streamlit run app/app.py`), keep Mock Mode ticked, sample photo on. For each case write expected and actual, and take a screenshot. TC-17 the yellow MOCK OUTPUT banner is shown above the numbers · TC-18 clear the prompt box completely and press Enter: a red error, no numbers · TC-19 prompt `cotton . weed .`: the GeoJSON status says "No weed zones approved" · TC-20 slider at 0.90: the screen shows 1 weed cluster and the GeoJSON has exactly 1 feature · TC-21 default settings: the downloaded GeoJSON has no `active_ingredient` field · TC-22 tick Force System 2 with Cotton: the status says MOCK DIAGNOSIS, never "verified" · TC-23 Force System 2 with Wheat, then Rice, then Sugar Beet: no Pyrithiobac anywhere · TC-24 change Crop Context: the prompt box changes to `weed . <crop> plant .` · TC-25 upload a text file renamed to `.jpg`: "Image Decoding Error" · TC-26 the GeoJSON metadata has a `coordinate_note` saying it is not GPS · TC-27 open a downloaded GeoJSON at `geojson.io`: count the shapes and compare with the weed clusters on screen.

Submit: Tab SU-3: `TC` · `steps` · `expected` · `actual` · `pass/fail` · `screenshot file`. Screenshots in `Drive/Sumant/SU-3`. A GitHub issue for every fail.
Done when: 11 rows with results and screenshots.

SU-4 · System tests in real mode (no GPU needed)

about 2 h

Why: The demo runs in real mode; we must know what a user sees and how long it takes on a normal laptop.

Steps:
1. Untick Mock Mode. The first run downloads about 1.6 GB of model weights; wait.
2. Run the sample photo, then one cotton photo from `data/datasets/cottonweeddet3_official/images/` (after SA-1′s build step, or ask Sahil for one).
3. For each run note: total seconds (use a phone stopwatch), the green REAL OUTPUT banner, the status text, how many weed clusters, any error.
4. Write 6 test cases (TC-28 to TC-33) for real mode from what you saw: what must be on screen, acceptable time, error messages.

Submit: Tab SU-4 with the runs and the 6 cases; screenshots in `Drive/Sumant/SU-4`.
Done when: Two real runs recorded and 6 cases written.

SU-5 · Run the crop-or-weed comparison yourself (Week 5 T2.5, first round)

about 2 h

Why: T2.5 decides which model checks uncertain plants. A second run on another machine shows whether the result repeats.

Steps:
1. Create a free Cloudflare account; in the dashboard open AI → Workers AI → "Use REST API"; copy the Account ID and create an API token with the Workers AI template. Put them in `.env` as `CLOUDFLARE_ACCOUNT_ID=` and `CLOUDFLARE_API_TOKEN=`.
2. Check: `python scripts/check_providers.py` must show two PASS lines.
3. After the build step of SA-1 (needs the Kaggle token): `python scripts/compare_crop_or_weed.py`. It takes about 10 minutes because Groq allows only a few image calls per minute.
4. Upload `data/outputs/t2_2/results.json` and copy the final table from the terminal.

Submit: `Drive/Sumant/SU-5/results.json` and the table in tab SU-5.
Done when: Your table posted; differences from Chaitanya's run (Groq 16/20, Clef 14/20, row rule 13/20) noted.

SU-6 · Report content: introduction and related work (Week 5 T2.7)

about 3 h

Why: Chaitanya writes chapters 1 and 2 from your notes.

Steps: Under heading SU-6 in the Report Content doc: (1) the problem in five bullets (blanket spraying, cost, why weed-versus-crop is hard) with a source for every number; (2) for each paper below: what they did, the dataset, the key number, one line on how we differ. Grounding DINO (Liu et al., ECCV 2024, arxiv.org/abs/2303.05499) · SAM 2 (Ravi et al., 2024, arxiv.org/abs/2408.00714) · Grounded SAM (Ren et al., 2024, arxiv.org/abs/2401.14159) · Milioto, Lottes, Stachniss (ICRA 2018, arxiv.org/abs/1709.06764) · Chong et al. (IROS 2025, github.com/PRBonn/WeedsAreWeird) · Nasir, Rehman, Hussain (Frontiers in Plant Science 2026, doi 10.3389/fpls.2025.1735096).

Submit: Heading SU-6 in the Report Content doc.
Done when: Five problem bullets and six paper summaries, each with its source.

5 Amit · tester (Week 4 + about 10% of Week 5)

AM-1 · Re-test and confirm the eight closed issues

about 2 h

Why: You found these bugs; you confirm the fixes. That is how the defect life cycle ends.

Steps:
1. `git pull`, then start the app.
2. Repeat **your own steps** from issues #6, #7, #8, #9, #10, #11, #12 and #13 (each issue page has the steps and Chaitanya's re-test comment).
3. On each issue add a comment: "Re-tested by tester on (date), Windows, commit (first 7 characters of `git log -1`): PASS" with a screenshot, or "FAIL" with what you saw. If it fails, click **Reopen**.

Submit: Eight GitHub comments with screenshots.
Done when: All eight issues have your comment.

AM-2 · Regression: repeat TC-01 to TC-16 on the new main

about 3 h

Why: Fixes can break other things. Re-running your Cycle 0 cases shows whether anything regressed.

Steps: Follow `docs/testing/System_Test_Runbook_Cycle0.pdf` Part C exactly as on 16 October, on the latest `main`. Note any case whose result changed and why.

Submit: New tab **AM-2** in the results sheet (same columns as Cycle 0); screenshots in `Drive/Amit/AM-2`; an issue for every new failure.
Done when: 16 rows with results.

AM-3 · Record the app in real mode (T4.5)

about 1 h

Why: We need a backup video of a real run in case the demo laptop fails.

Steps:
1. Untick Mock Mode and run one cotton or sugar beet photo (CPU is fine, only slow).
2. Record the screen (Windows: Win + G, Xbox Game Bar, "Start recording"): upload, wait, result with the green REAL banner, scroll through, download the GeoJSON.
3. Note the total time.

Submit: Video in `Drive/Amit/AM-3`, time in the group.
Done when: One complete real-mode recording.

AM-4 · Demo dry run (Week 5 work)

about 1 h

Why: The demo must not surprise us on the viva day.

Steps: Go through `docs/guides/04_Demo_Playbook.pdf` step by step on your laptop as if presenting. Mark every step that fails, is confusing, or no longer matches the app.

Submit: Tab AM-4: `step` · `ok / problem` · `what happened`.
Done when: Every playbook step has a row.

AM-5 · Slide deck skeleton (Google Slides)

about 1 h

Why: Each member will add 2–3 slides; one shared style avoids a patchwork deck.

Steps: Make a Google Slides deck with one clean style and these empty sections: Problem · Research question · Pipeline · Safety gate · Data and ground truth · Results · Testing · Limitations and future work · Team. Share edit access with the group.

Submit: Link in the group.
Done when: Deck exists with all sections.

6 Chaitanya and Claude (in parallel, for context)

Crop-protection metric and weed recall in `src/evaluation/` (tested with SA-2's numbers) · baseline real run over the 38 images on the laptop GPU (**Checkpoint 2, Sunday 25 October**) · loading the real ICAR knowledge store so prescriptions work in real mode · crop-exclusion improvement (T1.6) and triage before the map (T1.7) · Clef Flash inside the workflow (T2.8) · report and slides in Typst from your sheets · reviewing and merging all changes.

7 Templates and rules

7.1 GitHub issue for a defect

Title: `[TC-xx] what goes wrong, in one line`. Body: **Steps:** numbered, so anyone can repeat them · **Expected:** · **Actual:** · **Severity:** high (crop could be sprayed, mock shown as real, or the demo crashes) / medium

(wrong number or misleading screen) / low (cosmetic) · **Environment:** Windows/Linux, Python version, commit (`git log -1`), Mock or Real · **Evidence:** screenshot attached.

7.2 Rules

- No “done”, “verified” or “100%” without a screenshot or pasted output.
- The tester who filed a defect closes it; nobody closes their own fix.
- Do not edit `eval_manifest_v2`, change models or add packages.
- If an AI tool helps you, read and understand what it wrote before you submit it; you will be asked about it in the viva.